

Table of Contents

Guide 2: Running Jobs on Hydra (the Smithsonian HPC Cluster)	2
Prerequisites	2
Logging In	2
Where to Put Your Data	3
Modules: Loading Software	4
Submitting Your First Job	5
Monitoring Your Job	6
Interactive Jobs	7
Recommended Project Structure	7
Citing Hydra in Publications	8
Troubleshooting	8
Next Steps	9

Data Management for Ecology & Environmental Science

Overview

Ecological research produces heterogeneous data including field observations, sensor outputs, genomics, imagery, and spatial data. Effective data management ensures integrity, accessibility, and reusability across the project lifecycle.

Primary infrastructure:

- **Hydra** - institutional storage and computing
- **Globus** - data transfer and sharing

--

Data Organization & Naming

Recommended Directory Structure

Guide 2: Running Jobs on Hydra (the Smithsonian HPC Cluster)

Hydra is the Smithsonian's high-performance computing (HPC) cluster — a shared system of compute nodes you can use for analyses that won't run (or run too slowly) on a laptop. Genomic assembly, phylogenetic inference, large-scale image analysis, simulations, and bioacoustic processing are all common Hydra workloads at NZCBI.

Hydra is administered by the Office of the Chief Data Officer (under the Office of Digital and Innovation), the same office that maintains the Globus documentation linked elsewhere on this site.

This guide walks through everything you need to run your first analysis on Hydra: connecting, putting your data in the right place, loading the software you need, submitting a job, and getting your results back.

!!! note "This guide is a practical, NZCBI-flavored adaptation" The authoritative reference for Hydra is the [SI HPC Confluence wiki](#), particularly the [Quick Start Guide](#). A [complete 250+ page PDF](#) is also available. This guide aims to get NZCBI researchers from zero to first job submission with cross-links to the wiki for deeper details.

!!! tip "Want a hands-on workshop instead?" The Smithsonian Workshops [Hydra-introduction repository](#) has a full hands-on tutorial that walks through a real phylogenetics analysis (IQ-TREE) end-to-end. Most of the practical content here is adapted from that workshop and the official wiki.

Prerequisites

Before you can use Hydra, you need:

Requirement	How to get it
A Hydra account	Request via the SI Service Portal
A Hydra password	Set on first login via the password self-service (separate from your SI network password)
Either Smithsonian network access (on-site or VPN) or access to telework.si.edu	All Smithsonian staff have telework.si.edu access by default

!!! note "Hydra accounts are independent of SI network accounts" Even though your Hydra **username** is usually the same as your SI username (the part of your email before @si.edu, lowercase), your Hydra **password** is separate and managed independently. You'll set it the first time you log in.

Logging In

There are two ways to connect to Hydra. Use whichever fits your situation.

Option 1: telework.si.edu (works from anywhere)

The easiest way to get a terminal on Hydra is through the Smithsonian's telework portal — no VPN, no extra software, works from any browser.

1. Go to [telework.si.edu](#) and log in with your SI credentials.
2. Expand the **IT Tools** section.

3. Click **Hydra**.
4. Click one of the **Web SSH terminal (WeTTY)** links to open a web-based terminal in your browser.
5. At the login: prompt, enter your Hydra username (lowercase).
6. At the password: prompt, enter your Hydra password.

Option 2: Direct SSH (faster, requires Smithsonian network)

If you're on the Smithsonian network (on-site or via VPN), a direct SSH connection from your own terminal is faster and more flexible than the web terminal.

On Mac or Linux: open Terminal and run:

```
ssh {your-username}@hydra-login01.si.edu
```

On Windows: open Command Prompt (search "cmd" in the Start menu) and run the same command. If prompted about the authenticity of the host, type yes.

!!! note "Need a VPN?" SI staff can request VPN access through the [SI Service Portal](#). VPN is required for direct SSH from off the Smithsonian network.

Resetting your password

Hydra requires password changes every 180 days. To reset (initial setup, expired, or forgotten):

- **On telework:** open the Hydra option in IT Tools and choose **Password Self Help**.
- **On-site or VPN:** go to galaxy.si.edu.

Choose **Request an email with a password reset link**, enter your Hydra **username** (not your email address), and a reset link will be emailed to your institutional inbox.

Where to Put Your Data

Once you're logged in, you'll see a prompt that looks like:

```
[{your-username}@hydra-login01 ~]$
```

You're in your home directory. **Don't store project data in /home**. It has a small quota and isn't designed for analysis I/O. Hydra has three filesystem areas for project work:

Filesystem	Purpose	Notes
/scratch	Temporary working storage during analysis	Subject to scrubbing — see warning below
/data	Storage with backup	Most stable for project files you're not actively touching

!!! danger "Files inactive for 180+ days on /scratch are scrubbed" Your /scratch/public directory is scrubbed: any file that hasn't been **modified** in 180 days is automatically removed. This catches a lot of researchers off guard at the end of a long project. Either keep work active, move finished outputs to /data or off-cluster (e.g., via scp or [Globus](#)). See the [Disk Space and Disk Usage wiki](#) for full details.

For genomics and biology work — which covers most NZCBI computing — change into your scratch directory:

```
cd /scratch/public/genomics/{your-username}
```

Create a project directory

Once you're in your scratch space, the convention is one directory per project. This keeps your data, scripts, and results organized as you accumulate work over time.

```
mkdir my-first-project  
cd my-first-project
```

You can use whatever name fits — panda-genome-2026, nest-acoustics-pilot, etc.

Modules: Loading Software

Hydra has hundreds of scientific software packages installed system-wide. Rather than putting them all on your PATH at once (which would conflict), Hydra uses a **module** system: you load only the modules you need for a given session or job.

Browsing available software

To see everything installed:

```
module avail
```

The output is long. To narrow it down, search for a specific tool — for example, IQ-TREE:

```
module avail bioinformatics/iqtree
```

You can also browse the full list in your browser at the [Hydra module list](#).

Reading module help

Every module has a short help page describing what it provides and how to invoke it:

```
module help bioinformatics/iqtree
```

This shows the binary name (e.g. iqtree2), the version, and a link to the upstream documentation.

Loading a module

Loading a module makes its commands available in your current shell:

```
module load bioinformatics/iqtree
```

After this runs, iqtree2 is on your PATH and you can call it directly.

!!! tip “Pin the version in production work” module load bioinformatics/iqtree loads whatever the current default version is. For reproducibility, pin a specific version: module load bioinformatics/iqtree/2.1.3. The Hydra team occasionally changes which version is the default; pinning protects you from silently changing tools mid-project.

!!! note “What if my software isn't installed as a module?” You have a few options. You can compile from source in your own scratch space, or use [Conda](#) (which Hydra supports) to manage your own environments. Both approaches are documented on the SI HPC wiki. You can also email SI-HPC@si.edu to request that a package be installed system-wide.

Submitting Your First Job

Loading a module on the login node lets you *test* a command, but you should never run real analyses on the login node. Login nodes are shared by everyone connecting to Hydra; running a heavy job there slows everyone down and may get your process killed.

Real analyses go through the **scheduler**, which dispatches your job to a compute node with the resources you request. Hydra's scheduler is **Univa Grid Engine (UGE)**, and you interact with it through commands like `qsub` (submit), `qstat` (status), and `qacct` (completed-job info).

Step 1: Get your data onto Hydra

You have several options for moving files to Hydra. From easiest to most powerful:

Method	When to use
Globus	Best for large files or bulk transfers between Hydra and other storage endpoints (DAMS, STRI, your laptop). See Globus at the Smithsonian for details.
wget directly on Hydra	Best when your data is at a public URL (a journal supplement, an S3 bucket, NCBI, etc.) — saves the round-trip through your laptop.
scp from your laptop	Good for one-off transfers of a handful of small-to-medium files.
FileZilla	GUI alternative to <code>scp</code> . Connect to host <code>hydra-login02</code> , port 22, with your Hydra credentials.

For a full description of file transfer options on Hydra, see [Disk Space and Disk Usage → How To Copy](#) on the wiki.

For this guide we'll assume your input data is already on Hydra in `/scratch/public/genomics/{your-username}/my-first-project/`.

Step 2: Build a job submission script

A "job script" is a small shell script with extra header lines (called **directives**) that tell the scheduler what resources you need. Hydra provides a web tool to generate these correctly — strongly recommended for your first few jobs.

1. Open the [QSub Generation Utility](#) (also linked from telework's Hydra section). Use Chrome or Firefox; Safari is not recommended.
2. Fill in the form:
 - **CPU time:** short (good default for jobs under a few hours)
 - **Memory:** how much RAM your job needs (start with 4 GB if unsure)
 - **Type of PE:** multi-thread for jobs that use multiple cores on one node
 - **Number of CPUs:** how many cores to request (4 is a reasonable starting point)
 - **Shell:** `sh`
 - **Modules to add:** the modules your job needs (e.g. `bioinformatics/iqtree/2.1.3`)
 - **Job specific commands:** the actual commands you want to run (see example below)
 - **Job Name:** a short identifier (e.g. `iqtree`)
 - **Log File Name:** where to write standard output (e.g. `iqtree.log`)
 - **Change to CWD:** `Y`
 - **Join output & error files:** `Y`

- **Send email notifications:** Y, with your email address

A typical commands block for an IQ-TREE phylogenetics analysis looks like:

```
iqtree2 -s Exon_50per_taxa.phylip.txt \
        -nt $NSLOTS \
        -pre exon_50per_taxa
```

\$NSLOTS is automatically set to the number of CPUs you requested, so the program uses exactly the resources you asked for.

3. Click **Check if OK** to validate.
4. Click **Save it** to download the generated job file.
5. Rename the downloaded file to match your job name (e.g. `iqtree.job`) — the default `qsub.job` gets confusing fast.
6. Transfer the job file to Hydra into your project directory using whichever method you used for your input data.

!!! warning “Always start small” For a brand-new analysis, request modest resources (a few CPUs, a few GB of RAM, “short” queue) and verify the job runs end-to-end on a small test input *before* scaling up. Submitting a 24-hour, 64-CPU job that fails in the first 30 seconds because of a typo wastes both your time and the cluster’s.

Step 3: Edit the job file (optional)

You can edit job scripts directly on Hydra with a text editor. The friendliest is nano:

```
nano iqtree.job
```

Use the arrow keys to move the cursor. Save with **Ctrl+O**, then exit with **Ctrl+X**.

A common edit is pinning the module version. Find the line:

```
module load bioinformatics/iqtree
```

and change it to:

```
module load bioinformatics/iqtree/2.1.3
```

Step 4: Submit the job

```
qsub iqtree.job
```

You’ll see something like:

```
Your job 21884560 ("iqtree") has been submitted
```

The number is your **job ID** — write it down. You’ll use it for monitoring and troubleshooting.

Monitoring Your Job

Quick status check

```
qstat
```

This lists your active jobs (queued, running, etc.). If `qstat` shows nothing, your job either hasn’t been queued yet or has finished.

!!! warning “If your job disappears from `qstat` within seconds, it failed” A job that exits almost immediately is usually a sign of an error in the script — a missing module, a typo in a filename, a path that doesn’t exist. Check the log file before resubmitting.

Detailed status while running

```
qstat -j {job-id}
```

This shows everything the scheduler knows about a running job — current resource usage, node assignment, runtime, and any messages from the scheduler.

Reading the log file

The log file (whatever you named it in the QSub generator — `iqtree.log` in our example) is where your program's standard output and error messages go. You can read it while the job is running:

```
less iqtree.log
```

Use the spacebar to page through, q to quit.

Post-mortem on a completed job

After a job finishes, `qstat` no longer shows it. Use `qacct+` for the post-mortem:

```
qacct+ -j {job-id}
```

The most important line in the output is **maxvmem** — the peak memory your job actually used. Compare it to what you requested:

- If **maxvmem** was much less than what you requested, you're wasting cluster resources and should request less next time.
- If **maxvmem** came close to your request, your job will fail with an out-of-memory error if its dataset grows. Bump the request next time.

Interactive Jobs

Sometimes you want to run something interactively on a compute node — exploratory analysis, debugging a script that's failing, or testing parameters. For these, use `qrsh` instead of `qsub`:

```
qrsh -pe mthread 2
```

This drops you into a shell on a compute node with 2 cores allocated. From there you load modules and run commands directly, just as you would on the login node, but with proper compute resources.

!!! warning “qrsh puts you back in /home” When `qrsh` opens the compute-node shell, your working directory resets to `/home/{your-username}`. Always `cd` back to your project directory before running anything:

```
```bash
cd /scratch/genomics/{your-username}/my-first-project
```
```

When you're finished, exit the interactive job with:

```
exit
```

Recommended Project Structure

After you've submitted a few jobs, your project directory accumulates files: input data, job scripts, output files, log files. A consistent structure makes it possible to revisit a project months later without confusion.

A reasonable starting structure:

```
my-first-project/
├── data/
│   ├── raw/          # original input data, never modified
│   └── results/       # outputs from your analyses
├── jobs/             # .job submission scripts
└── logs/             # log files from completed jobs
```

You can scaffold this in one command:

```
mkdir -p data/raw data/results jobs logs
```

Then visualize it with tree (also a Hydra module):

```
tree
```

Adapt the structure as your project grows. Common additions are `scripts/` for analysis code, `notebooks/` for Jupyter work, and `docs/` for project notes.

Citing Hydra in Publications

If your published work used computations performed on Hydra, the SI HPC team asks that you cite the cluster. Suggested citation text:

Some of the computations in this paper were conducted on the Smithsonian High Performance Cluster (SI/HPC), Smithsonian Institution. <https://doi.org/10.25572/SHPC>

This citation supports continued investment in shared HPC resources at SI.

Troubleshooting

Login problems

Issue: “Permission denied” or “Password incorrect” on first login

- **Cause:** Your Hydra password hasn’t been set yet. Hydra passwords are separate from SI network passwords.
- **Solution:** Use the [password self-service](#) (or telework’s “Password Self Help” link) to set an initial password.

Issue: SSH connection times out from off-site

- **Cause:** Direct SSH only works from inside the Smithsonian network.
- **Solution:** Either connect to the SI VPN first, or use telework.si.edu instead.

Job problems

Issue: Job fails immediately, log file is empty or contains a “command not found” error

- **Cause:** The required module didn’t load, or the module name in your job script has a typo.
- **Solution:** On the login node, run `module avail` to verify the exact module name; reload the corrected name in your job script.

Issue: Job runs for a while then fails with “Eqw” or memory errors

- **Cause:** Your job exceeded its requested memory.

- **Solution:** Run `qacct+ -j {job-id}` on the failed job, look at `maxvmem`, and increase the memory request in your next submission.

Issue: Job sits in the queue for a long time

- **Cause:** The cluster is busy, or you’ve requested an unusual combination of resources (very high memory, many CPUs, etc.).
- **Solution:** Check the [cluster status page](#) to see overall load; consider reducing your request if it’s larger than necessary.

Storage problems

Issue: “Disk quota exceeded” when writing to /home

- **Cause:** /home has a small quota and is not for project data.
- **Solution:** Move your project to /scratch, /pool, or /data depending on its lifecycle. See the [Disk Space and Disk Usage wiki](#) for quotas on each filesystem.

Issue: Files I created weeks ago have disappeared from /scratch

- **Cause:** /scratch and /pool are scrubbed — files unmodified for 180+ days are deleted.
- **Solution:** For long-term storage, move files to /data (which is intended for stable storage) or off-cluster via Globus. To prolong files on /scratch, touch them periodically or keep your work active.

Issue: Can’t find or write to /scratch/genomics/{username}

- **Cause:** Your scratch directory may not have been created when your account was provisioned.
- **Solution:** Email SI-HPC@si.edu and ask them to create it.

Next Steps

Get your results back to your laptop

Once your job has produced output files, you’ll want to move them off Hydra for analysis, sharing, or archival. The same transfer methods that work for getting data *onto* Hydra also work for getting it *off* — see [Globus at the Smithsonian](#) for the recommended approach for any non-trivial dataset.

Learn the hands-on workshop

If you want a guided exercise that walks you through everything in this guide on a real dataset, work through the [Smithsonian Workshops Hydra-introduction repository](#). It includes the IQ-TREE example used here, with sample data and step-by-step verification.

Explore the official wiki

The [Smithsonian HPC Confluence wiki](#) is the authoritative reference for Hydra. Especially useful pages:

- [Quick Start Guide](#) — the official version of this material
- [Disk Space and Disk Usage](#) — quotas, filesystem map, scrubbing details
- [Conda tutorial](#) — installing your own software environments
- [Submitting jobs](#) — deep dive on UGE directives and job scripts
- [Module list](#) — every package available system-wide
- [Cluster status](#) — current queue and node availability

The complete documentation (250+ pages) is also available as a downloadable PDF, linked from the [HPC home page](#).

Get help

| For | Contact |
|---|--|
| Hydra-specific questions, account issues, software install requests | SI-HPC@si.edu |
| Data transfer (Globus) questions | SI-Globus@si.edu |
| NZCBI-specific computing questions | stabachj@si.edu |

The SI HPC team also runs regular **Brown Bag** sessions where you can bring questions about specific analyses or pipelines. Watch [SI-HPC](#) announcements for the schedule.

This guide is adapted from the [SI HPC Quick Start Guide](#) and the [Smithsonian Workshops Hydra-introduction](#) workshop materials, with NZCBI-specific framing and cross-links to the Globus documentation. .